Lab 3 - Component Modelling & Architectural Pattern Selection

Student Club Event Ticketing & Budget Portal

NAME: I.V. Thanvitha

SRN: PES2UG24CS197

SECTION: D

1. Scenario Review

Key requirements: Club Leads submit event proposals and request budgets; the system routes approvals sequentially through Faculty Coordinator -> Finance Office -> Dean, generates secure QR-code tickets, and supports fast ticket validation at venue entry. The main design challenges are access control, approval-data consistency, secure ticket authenticity, and low-latency check-in.

2. Architectural Style Analysis

Style	Assessment	Fit for this portal
Layered	Selected	Clear separation between UI, business rules and data access; simple to maintain and well suited to one integrated campus portal.
Microservices	Not selected	Independent scaling is useful, but network calls, deployment overhead and distributed-data complexity are unnecessary for this scope.
Client-Server	Not selected	Centralized control is simple, but it gives weaker separation of business modules and can create a server bottleneck as features grow.

3. Architecture Selection

We chose Layered Architecture for the Student Club Event Ticketing & Budget Portal.

- Reason 1 - Separation of concerns: the Presentation Layer handles Club Lead, Campus Admin and venue check-in screens; the Business Layer handles authentication, event/budget workflow, ticket generation and validation; the Data Layer stores users, events, budgets, approvals and tickets. This makes the system easier to test, maintain and modify.
- Reason 2 - Consistent workflow and shared data: budget approvals, event status and ticket state are strongly related. A centralized business/data structure allows sequential approval rules and ticket status updates to remain consistent without distributed transaction complexity.

4. Security Advantage

Authentication and RBAC are centralized in the Business Layer. Every protected operation is checked before access to event, budget or ticket data. A Club Lead cannot perform Campus Admin actions, and QR validation can reject forged, revoked or already-used tickets. Direct presentation-to-database access is not allowed.

5. Performance Benefit

Layered in-process service calls avoid the extra network hops of a microservices design. The QR Check-In Validation component can perform a local signature/authenticity check followed by an indexed ticket-status lookup, supporting the required sub-100 ms venue validation target under normal peak-load design assumptions.

6. Components & Key Interfaces Shown in the Diagram

Components: Web Portal UI; Venue Check-In UI; Authentication & RBAC; Event & Budget Workflow; Ticketing & QR Generation; QR Check-In Validation; Notification Service; Repository / Data Access; Portal Database. **Interfaces:** IAuthentication, IEventBudget, ITicketing, IValidateTicket, IUserRepository, IEventBudgetRepository, ITicketRepository, ITicketLookup, INotification and IDatabase. Provided/required interface notation and data-flow labels are shown in the component diagram.